

Runtime Systems

Exceptions, Memory Management, GC

Liting Zhang

University of West Florida

- 1 Overview
- 2 Exceptions and error handling
- 3 Unwinding and cleanup
- 4 Memory management strategies
- 5 Rust : ownership
- 6 Industry angle
- 7 Review

Overview

Topics today:

- Exception handling models: error codes, exceptions, typed results
- Stack unwinding, finally blocks, resource cleanup
- Memory management: manual free, reference counting, tracing GC
- Industry angle: reliability, memory safety, debugging failures
- Rust: ownership as an alternative to tracing GC

Languages:

- Java, C++, TypeScript, Rust
- Python for reference counting and context managers

Learning goals

After this lecture, you should be able to:

- Compare error codes, exceptions, and typed results
- Trace stack unwinding and predict which cleanup code runs
- Compare manual memory, reference counting, and tracing GC
- Explain why GC does not prevent all memory leaks
- Connect runtime choices to reliability and performance

What a runtime system is responsible for

A runtime system provides the machinery and rules for:

- function calls, call stack, stack frames
- exception propagation and unwinding
- heap allocation and object layout
- memory reclamation strategy
- observability: stack traces, crash reports, profiling hooks

Takeaway:

- language features often depend on runtime support and conventions

Two kinds of failures in real systems

In production:

- expected: timeouts, missing files, invalid input, permission denied
- unexpected: null dereference, out-of-bounds, use-after-free

Runtime choices determine:

- how failures are represented
- how they propagate across layers
- whether cleanup happens reliably during failure paths

Practice: expected vs unexpected

Classify each as expected or unexpected:

- a user enters an invalid email address
- a network call to a dependency times out
- an array index goes out of range
- a null pointer is dereferenced
- a file is missing in a CLI tool

Practice answer: expected vs unexpected

One reasonable classification:

- invalid email address: expected
- network timeout: expected
- array index out of range: unexpected (bug or missing guard)
- null pointer dereference: unexpected (bug)
- missing file in CLI tool: expected

Takeaway:

- expected failures should be part of normal control flow
- unexpected failures should be loud and easy to diagnose

- 1 Overview
- 2 Exceptions and error handling
- 3 Unwinding and cleanup
- 4 Memory management strategies
- 5 Rust : ownership
- 6 Industry angle
- 7 Review

Error-handling models engineers choose from

Common models:

- error codes: return a status and check it
- exceptions: throw and catch
- typed results: return a value like Result or Option

Questions engineers ask:

- is the failure expected and recoverable
- should handling be forced by the compiler
- how much overhead is acceptable on the fast path
- how do we attach context and log consistently

Error codes: strengths and weaknesses

Strengths:

- explicit at the call site
- predictable control flow
- works well across language boundaries

Weaknesses:

- easy to forget to check
- boilerplate and deeply nested if checks
- cleanup can be duplicated across error paths

Exceptions: strengths and weaknesses

Strengths:

- reduces boilerplate in normal code
- propagates errors to a handler automatically
- integrates naturally with stack traces

Weaknesses:

- hidden control flow
- cleanup must be structured or it is skipped
- teams often need conventions about what can throw

Typed results: strengths and weaknesses

Strengths:

- failure is part of the return type
- callers can be forced to handle both cases
- branching on error types is straightforward

Weaknesses:

- verbose signatures if everything returns Result
- developers may still shortcut with unwrap or crash

Checked vs unchecked exceptions

Java separates exceptions into categories:

- checked: must be caught or declared, often for recoverable failures
- unchecked: programming errors and runtime failures

Design tension:

- checked exceptions document and enforce handling
- but can encourage catch-all patterns if overused

Java: checked exception forces handling

The call site must catch or declare.

```
1  import java.io.*;
2
3  class Demo {
4      static String readFirstLine(String path) throws IOException {
5          try (BufferedReader br = new BufferedReader(new
6              ↪   FileReader(path))) {
7              return br.readLine();
8          }
9      }
10
11     public static void main(String[] args) {
12         try {
13             System.out.println(readFirstLine("data.txt"));
14         } catch (IOException e) {
15             System.out.println("could not read file");
16         }
17     }
```

TypeScript and Python error style

In many dynamic ecosystems:

- exceptions are the default failure channel
- types do not describe what is thrown
- async adds rejected promises as a common failure path

Practical convention:

- catch at boundaries and translate to stable error responses

TypeScript: synchronous throw vs async failure

Both patterns exist in typical code.

```
1 function parsePort(s: string): number {  
2     const n = Number(s);  
3     if (!Number.isInteger(n) || n <= 0) {  
4         throw new Error("bad port");  
5     }  
6     return n;  
7 }  
8  
9 async function fetchJson(url: string): Promise<any> {  
10     const res = await fetch(url);  
11     if (!res.ok) {  
12         throw new Error("http error");  
13     }  
14     return await res.json();  
15 }
```

Practice: choose a model

For each scenario, pick a reasonable primary error model:

- parsing user input in a web request
- network timeout calling a dependency
- invalid configuration at startup
- out-of-bounds access in a core data structure
- missing optional file in a CLI tool

Practice answer: choose a model

One reasonable set of choices:

- parsing user input: typed result or exception translated to a 400 response
- network timeout: typed error propagated, handled at boundary with retry policy
- invalid config at startup: fail fast, crash with clear message
- out-of-bounds: bug, crash/panic in development, add guards where needed
- missing optional file: typed error or exception caught at main with user-friendly output

- 1 Overview
- 2 Exceptions and error handling
- 3 Unwinding and cleanup**
- 4 Memory management strategies
- 5 Rust : ownership
- 6 Industry angle
- 7 Review

Stack unwinding

When an exception is thrown:

- the runtime searches for a handler up the call stack
- frames are popped until a matching handler is found
- cleanup code runs based on language rules

Key question:

- what is guaranteed to run during unwinding

Finally blocks and structured cleanup

Cleanup must happen even on early exit:

- normal return
- thrown exception
- multiple returns

Constructs:

- Java: finally, try-with-resources
- Python: try/finally, with
- C++: destructors and RAI
- Rust: Drop at end of scope

Java: finally runs during unwinding

Predict the output.

```
1 class T {  
2     static void f() {  
3         try {  
4             System.out.println("A");  
5             throw new RuntimeException("boom");  
6         } finally {  
7             System.out.println("B");  
8         }  
9     }  
10  
11     public static void main(String[] args) {  
12         try { f(); } catch (Exception e) { System.out.println("C");  
13             ↪ }  
14     }  
15 }
```

Practice answer: finally runs during unwinding

Expected output:

- A prints in the try block
- exception thrown, then finally runs, so B prints
- caller catches, so C prints

Takeaway:

- structured cleanup makes failure paths safe by default

C++ cleanup: RAII mental model

In C++:

- local objects are destroyed when leaving scope
- destruction happens on normal return and on exception unwinding
- resource cleanup is encoded in destructors

Typical consequence:

- less explicit finally code
- more emphasis on wrapping resources in objects

C++: RAII closes files on all paths

Cleanup is deterministic at scope exit.

```
1  #include <fstream>
2  #include <string>
3  #include <stdexcept>
4
5  std::string read_all(const std::string& path) {
6      std::ifstream in(path);
7      if (!in) { throw std::runtime_error("open failed"); }
8
9      std::string s, line;
10     while (std::getline(in, line)) {
11         s += line;
12         s += "\n";
13     }
14     return s; // in is closed here
15 }
```

Resource leaks are not only memory leaks

Common resources:

- file descriptors, sockets, database connections
- locks, permits, thread handles

Production symptom:

- memory stable, but process dies due to too many open files

Practice idea:

- prefer language-supported structured cleanup constructs

Practice: where should you catch

For each place, decide catch or do not catch:

- inside a low-level parsing function
- inside a reusable library utility
- at the HTTP request handler boundary
- at a background job loop boundary
- at thread entry points

Practice answer: where should you catch

One reasonable policy:

- low-level parsing function: do not catch, return/propagate with context
- reusable library utility: do not catch unless translating to a better error type
- HTTP request handler boundary: catch and translate to an error response
- background job loop boundary: catch, log, keep worker alive
- thread entry points: catch, log, avoid silent thread death

- 1 Overview
- 2 Exceptions and error handling
- 3 Unwinding and cleanup
- 4 Memory management strategies**
- 5 Rust : ownership
- 6 Industry angle
- 7 Review

The memory management problem

Programs allocate memory for:

- stack: locals and call frames
- heap: dynamic objects with flexible lifetimes

Key questions:

- who owns a heap allocation
- when it can be reclaimed
- how to prevent dangling pointers and leaks

Manual allocation and free

Manual memory management:

- allocate with malloc/new
- free with free/delete

Benefits:

- explicit control over lifetime and layout
- predictable behavior in tight loops when carefully engineered

Costs:

- use-after-free, double-free, memory leaks
- complex ownership reasoning at scale

C++: raw ownership creates easy mistakes

The caller must remember the ownership rule.

```
1  #include <cstring>
2
3  char* make_msg(const char* name) {
4      char* p = new char[64];
5      std::strcpy(p, "hi ");
6      std::strcat(p, name);
7      return p; // caller must delete[] p
8  }
```

C++ practice: what goes wrong

Identify two different bugs that are easy to write with `make_msg`:

- one that causes a leak
- one that causes a crash or memory corruption

C++ practice answer: what goes wrong

Two common bug shapes:

- leak: caller forgets to delete[] the returned pointer
- crash/corruption: caller deletes twice, or uses the pointer after deleting

Takeaway:

- raw ownership is hard to audit across module boundaries

Modern C++ approach: RAI and smart pointers

Modern C++ conventions:

- prefer `std::string`, `std::vector` for buffers
- prefer `std::unique_ptr` for single-owner heap allocations
- use `std::shared_ptr` only when shared ownership is truly needed

Goal:

- make ownership visible in types, not in comments

C++: `unique_ptr` makes ownership explicit

Deterministic cleanup at scope exit.

```
1  #include <memory>
2  #include <string>
3
4  std::unique_ptr<std::string> build() {
5      auto p = std::make_unique<std::string>("hello");
6      return p; // moved out, still one owner
7  } // no manual delete needed
```

Reference counting

Reference counting strategy:

- each object tracks how many references point to it
- increment on new reference, decrement on drop
- free when the count hits zero

Benefits:

- reclamation is incremental
- predictable in many interactive workloads

Problem:

- cycles can leak: A points to B and B points to A

Tracing garbage collection

Tracing GC reclaims by reachability:

- identify roots: stack, globals, registers
- mark objects reachable from roots
- reclaim unmarked objects, sometimes compact

Key property:

- cycles are collected if unreachable from roots

Generational garbage collection

Common observation:

- most objects die young

Design:

- collect young objects frequently
- collect old objects less frequently
- track old-to-young references with write barriers

GC does not prevent all leaks

GC collects unreachable objects, but leaks still happen when:

- objects remain reachable from roots but are logically unused
- caches grow without eviction
- event listeners or callbacks retain closures

Symptom:

- heap growth, more GC cycles, worse tail latency

TypeScript: accidental retention sketch

A cache can grow forever without eviction.

```
1  const cache = new Map<string, any>();
2
3  export async function getUser(id: string) {
4    if (cache.has(id)) return cache.get(id);
5
6    const user = await fetch(`/api/user/${id}`).then(r =>
7      ↪ r.json());
8    cache.set(id, user);
9    return user;
10 }
```

Practice: find the leak

In the cache example, list two fixes:

- one policy fix
- one technical fix

Also name one place where the bug might show up:

- latency, memory, or error rate

Practice answer: find the leak

Fixes:

- policy fix: only cache a bounded set of ids, or cache only hot keys
- technical fix: add eviction (LRU), TTL, size cap, or periodic cleanup

Symptom:

- memory increases, GC becomes more frequent, tail latency increases

- 1 Overview
- 2 Exceptions and error handling
- 3 Unwinding and cleanup
- 4 Memory management strategies
- 5 Rust : ownership**
- 6 Industry angle
- 7 Review

Rust : ownership as alternative to tracing GC

Rust enforces lifetime and aliasing rules:

- each value has an owner
- references must not outlive the owned value
- moves transfer ownership instead of copying by default

Takeaway:

- many memory safety bugs become compile-time errors

Rust: moves prevent double ownership

Ownership moves by default.

```
1 fn main() {  
2     let s = String::from("hi");  
3     let t = s;           // moved  
4     // println!("{}", s); // compile error  
5     println!("{}", t);  
6 }
```

Rust: Result for recoverable errors

Rust splits failure into:

- Result for recoverable errors
- panic for programmer bugs or impossible states

Practical benefit:

- error handling is explicit in types, but still concise with ? syntax

Rust: Result and ? for propagation

Errors are values, propagation is concise.

```
1 use std::fs::File;
2 use std::io::{self, Read};
3
4 fn read_all(path: &str) -> Result<String, io::Error> {
5     let mut f = File::open(path)?;
6     let mut s = String::new();
7     f.read_to_string(&mut s)?;
8     Ok(s)
9 }
```

Practice: panic or Result

For each case, choose panic or Result:

- parse a user-provided integer
- open a config file at startup
- internal invariant violated in a data structure
- network timeout talking to a dependency
- index into an array with a user-provided index

Practice answer: panic or Result

One reasonable set of choices:

- parse user integer: Result
- open config file at startup: Result, but exit if missing and required
- internal invariant violated: panic
- network timeout: Result
- user-provided index: Result or safe check returning None/Err

- 1 Overview
- 2 Exceptions and error handling
- 3 Unwinding and cleanup
- 4 Memory management strategies
- 5 Rust : ownership
- 6 Industry angle**
- 7 Review

Industry angle: memory safety and reliability

Why runtime choices matter:

- memory safety bugs can become security issues
- crashes and leaks increase incident rate
- tail latency is affected by pauses and allocator pressure

Typical tradeoffs:

- C/C++: control and speed, more risk without strict discipline
- GC languages: simpler memory ownership, less control over reclamation timing
- Rust: strong safety guarantees, some complexity in ownership design

Choosing a strategy in practice

Questions engineers ask:

- is predictable latency required, or is average throughput the priority
- do we interface with native code and manual allocators
- what tooling is available for debugging leaks and corruption
- how costly are crashes in this environment
- what is the team skill set and existing ecosystem

Practical reality:

- many systems mix languages and therefore mix runtime strategies

Practice: choose a runtime style for components

For each component, pick a likely style and justify:

- browser UI code
- kernel driver
- backend web service handling many requests
- game engine core loop
- data science notebook workflow

Practice answer: choose a runtime style for components

One reasonable set of choices:

- browser UI: GC, focus on pause control and leak avoidance
- kernel driver: manual memory, strict review and sanitizers
- backend web service: GC or Rust, focus on reliability and tail latency
- game engine loop: C++ with RAII, custom allocators, predictable frame times
- data science notebook: GC or RC, fast iteration and rich libraries

- 1 Overview
- 2 Exceptions and error handling
- 3 Unwinding and cleanup
- 4 Memory management strategies
- 5 Rust : ownership
- 6 Industry angle
- 7 Review**

Review

- 1 Overview
- 2 Exceptions and error handling
- 3 Unwinding and cleanup
- 4 Memory management strategies
- 5 Rust : ownership
- 6 Industry angle
- 7 Review**

Summary: what to remember

- Error codes, exceptions, and typed results trade visibility for convenience
- Stack unwinding changes control flow and requires structured cleanup
- RAII, try-with-resources, with, and Drop make cleanup reliable
- Manual memory offers control but risks corruption and leaks
- Reference counting is incremental but cycles require special handling
- Tracing GC collects cycles but can introduce pauses and overhead
- GC languages can still leak through accidental retention from roots
- Rust ownership provides memory safety without tracing GC for many cases

Review questions

- What is stack unwinding and what code runs during it
- When are checked exceptions helpful and when are they harmful
- Why do cycles leak under reference counting
- Why does generational GC help performance in many programs
- Name two ways a GC language can still leak memory
- In Rust, when should you use Result versus panic